AV2 Codec ISO Media File Format Binding

Working Group Draft, 19 October 2026



This version:
https://aomediacodec.github.io/av2-isobmff/v1.0.0.html

Issue Tracking:
GitHub

Editors:
TBD
TBD

Warning

This specification is still at draft stage and should not be referenced other than as a working draft.

Copyright 2026, Alliance for Open Media

Licensing information is available at https://www.aomedia.org/license/

The MATERIALS ARE PROVIDED “AS IS.” The Alliance for Open Media, its members, and its contributors expressly disclaim any warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to the materials. The entire risk as to implementing or otherwise using the materials is assumed by the implementer and user. IN NO EVENT WILL THE ALLIANCE FOR OPEN MEDIA, ITS MEMBERS, OR CONTRIBUTORS BE LIABLE TO ANY OTHER PARTY FOR LOST PROFITS OR ANY FORM OF INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS DELIVERABLE OR ITS GOVERNING AGREEMENT, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND WHETHER OR NOT THE OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Abstract

This document specifies the storage format for [AV2] bitstreams in [ISOBMFF] tracks as well as in [CMAF] files.

Table of Contents

Pages

1 Terminology . . . 4

2 Bitstream features overview . . . 4

3 Basic Encapsulation Scheme . . . 5

3.1 General Requirements & Brands . . . 6

3.2 AV2 Sample Entry and Configuration . . . 6

3.2.1 AV2 Sample Entry . . . 6

3.2.2 AV2 Codec Configuration Box . . . 7

3.2.2.1 config_obu . . . 7

3.3 AV2 Sample Format . . . 9

3.3.1 General . . . 9

3.3.2 OBUs in samples . . . 10

3.3.3 Sync sample . . . 10

3.3.4 Layers . . . 11

3.3.5 AV2 sample reconstruction . . . 13

4 CMAF AV2 track format . . . 14

5 Common Encryption . . . 14

5.1 General Subsample Encryption constraints . . . 15

6 Codecs Parameter String . . . 16

Index . . . 17

Terms defined by this specification . . . 17

Terms defined by reference . . . 18

References . . . 20

Normative References . . . 20

List of Figures

Figure 3.1: List of AV2 OBU types and where they can be found . . . 6

Figure 3.2: Example mapping of AV2 temporal units into the ISOBMFF sample format. . . . 7

1. Terminology

The key words "**MUST**", "**MUST NOT**", "**REQUIRED**", "**SHALL**", "**SHALL NOT**", "**SHOULD**", "**SHOULD NOT**", "**RECOMMENDED**", "**NOT RECOMMENDED**", "**MAY**", and "**OPTIONAL**" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

All of the text of this specification is normative except sections explicitly marked as non-normative, examples, and notes.

Examples in this specification are introduced with the words "for example" or are set apart from the normative text with `class="example"`, like this:

> EXAMPLE 1 ¶
> This is an example of an informative example.

Informative notes begin with the word "Note" and are set apart from the normative text with `class="note"`, like this:

> NOTE: This is an example of an informative note.

2. Bitstream features overview

An AV2 bitstream is composed of a sequence of OBUs (Open Bitstream Units), grouped into Temporal Units. A Temporal Unit consists of a Temporal Delimiter OBU and all following OBUs up to, but not including, the next Temporal Delimiter OBU. Each Temporal Unit is associated with a single presentation time.

Every OBU begins with a 1- or 2-byte header that carries the obu_type and the temporal layer identifier obu_tlayer_id. When the OBU header extension is present (i.e. when obu_header_extension_flag is equal to 1 in that header), two additional layer identifiers are carried: the embedded layer identifier obu_mlayer_id and the extended layer identifier obu_xlayer_id. Depending on its type, an OBU can carry configuration information (e.g., a Sequence Header OBU or Layer Configuration Record OBU), metadata, or coded video data.

A Temporal Unit **MAY** contain multiple coded pictures but only one is output. AV2 defines a number of coded-frame OBU types, including Closed Loop Key OBUs, Open Loop Key OBUs, regular and leading tile-group OBUs, Switch OBUs, `OBU_RAS_FRAME`, Bridge Frame OBUs, and TIP (Temporally Interpolated Prediction) frames.

Frames that can be decoded without reference to other frames are Key Frames (carried in a Closed Loop Key OBU or an Open Loop Key OBU) and Intra frames. All other coded frames - including Inter Frames, Switch Frames, RAS frames, Bridge frames, TIP frames, and frames with show_existing_frame equal to 1 - have coding dependencies on other frames in the same or previous Temporal Units.

A Closed Random Access is the random access process that applies to an extended layer whose first coded frame is a Key Frame carried in a Closed Loop Key OBU. It starts a new coded video sequence for the extended layer; the layer's reference frame buffers are invalidated and every subsequent frame of that layer can be decoded. This is the primary random-access mechanism.

An Open Random Access is the random access process that applies to an extended layer whose first coded frame is a Key Frame carried in an Open Loop Key OBU. During sequential decoding, an Open Random Access point does not start a new coded video sequence; however, when a decoder initiates decoding at that point, the decoding process is treated as if it were the start of a new coded video sequence for the extended layer. Leading frames that follow an Open Random Access point **MAY** reference frames that precede it, and are discarded when decoding begins at the Open Random Access point.

In addition to Key Frames, AV2 provides two further mechanisms that support random access or stream switching: a RAS frame (`OBU_RAS_FRAME`) is an inter-predicted frame that uses long-term reference frames and enables random access or switching without inserting a full key frame; a Switch Frame (`OBU_SWITCH`) is an inter-predicted frame that enables switching between representations at a given point, typically in adaptive streaming scenarios.

AV2 supports three scalability dimensions: temporal layers (identified by obu_tlayer_id), embedded layers (identified by obu_mlayer_id), and extended layers (identified by obu_xlayer_id). The reserved obu_xlayer_id value `GLOBAL_XLAYER_ID` (equal to 31, as defined in [AV2]) denotes global scope and is used by OBUs that do not belong to any specific extended layer (such as the Temporal Delimiter OBU). A bitstream that uses a single distinct non-global value of obu_xlayer_id is a Singlestream bitstream; a bitstream that uses two or more distinct non-global values is a Multistream bitstream.

AV2 defines two distinct Metadata OBU types for carrying metadata: `OBU_METADATA_SHORT`, using the metadata short OBU syntax, and `OBU_METADATA_GROUP`, using the metadata group OBU syntax. A separate OBU type carries film grain synthesis parameters; its presence is indicated by the `film_grain_params_present` flag in the Sequence Header OBU.

The Sequence Header OBU also carries the bitstream's profile, level, and - when applicable - tier, as defined in Annex A of [AV2].

The Sequence Header OBU carries a `monotonic_output_order_flag` that defines the output mode of the associated coded video sequence.

When `monotonic_output_order_flag` is equal to 1, the output order of coded output frame equals their decoding order within the coded video sequence. Within a sample, the composition time (CTS) of the associated output frame is equal to the sample's decoding time (DTS); the sample **MAY** additionally contain hidden frames.

When `monotonic_output_order_flag` is equal to 0, the output order of coded output frames **MAY** differ from their decoding order. The composition time of each sample is then derived from a 'ctts box that signals the offset between the sample's decoding time (DTS) and the composition time of its associated output frame. As in the monotonic case, a sample **MAY** additionally contain hidden frames.

> NOTE: Non-monotonic output mode is new in AV2; AV1 supports only the monotonic mode.

§ 3. Basic Encapsulation Scheme

This section describes the basic data structures used to signal encapsulation of AV2 bitstreams in [ISOBMFF] containers.

There are two locations where OBUs can be found in a file conformant to this specification:

- Some High-Level Syntax OBUs, such as the Sequence Header OBU, are carried in the AV2CodecConfigurationBox as described in § 3.2 AV2 Sample Entry and Configuration. This corresponds to "availability through external means" as described in the [AV2] specification. These OBUs apply to all samples of the corresponding AV2SampleEntry.
- The other OBUs are stored in samples. This corresponds to "presence in the bitstream" as described in the [AV2] specification.

An ***AV2 sample*** is the union of a sample inclusive of the OBUs it carries, the OBUs carried in the AV2CodecConfigurationBox of the corresponding AV2SampleEntry, and an implicit Temporal Delimiter OBU.

> NOTE: This specification refers to samples as defined in [ISOBMFF], which are not to be mistaken with samples or sample values as defined in the [AV2] specification.

§ 3.1. General Requirements & Brands

A file conformant to this specification satisfies the following:

- It SHALL conform to the normative requirements of [ISOBMFF]
- It SHALL have the ***'av02'*** brand among the compatible brands array of the FileTypeBox
- It SHALL contain at least one track using an AV2SampleEntry, possibly transformed by encryption as specified in § 5 Common Encryption
- It SHOULD indicate a structural ISOBMFF brand among the compatible brands array of the FileTypeBox, such as 'iso6
- It SHALL contain at least one AV2 sample
- Each AV2 sample it contains SHALL form exactly one [AV2] Temporal Unit if reconstructed as described in § 3.3.5 AV2 sample reconstruction

§ 3.2. AV2 Sample Entry and Configuration

§ 3.2.1. AV2 Sample Entry

The ***AV2SampleEntry*** extends VisualSampleEntry and signals a track carrying AV2 samples.

```
class AV2SampleEntry extends VisualSampleEntry('av02') {
  AV2CodecConfigurationBox config;
}
```

An AV2SampleEntry SHALL contain exactly one AV2CodecConfigurationBox.

An AV2SampleEntry SHALL directly contain exactly one 'colr box with the 'nclx 'colour_type. Each value among `colour_primaries`, `transfer_characteristics`, and `matrix_coefficients` carried in that 'colr box, for the AV2 sample layers or operating points it applies to:

- SHOULD equal the corresponding effective value of the AV2 bitstream carried by the track if the latter is anything but 2 (`Unspecified`);

- SHALL take precedence over the corresponding effective value of the AV2 bitstream carried by the track.

The value of `full_range_flag` carried in that ‘colr box, for the AV2 sample layers or operating points it applies to, SHOULD equal the corresponding effective value of the AV2 bitstream carried by the track and SHALL take precedence over the corresponding effective value of the AV2 bitstream carried by the track.

An AV2SampleEntry SHALL directly contain exactly one ‘pixi box. ‘pixi boxes in an AV2SampleEntry SHALL have the least significant bit of ‘px_flags’ enabled. The constraints described in § 3.3.4 Layers apply to that AV2SampleEntry.

> NOTE: An AV2SampleEntry may directly contain additional boxes permitted by [ISOBMFF] for a VisualSampleEntry, such as ‘colr with a ‘colour_type value other than ‘nclx, ‘pasp, ‘clli, ‘mdcv or ‘btrt. They may be located before and/or after the AV2CodecConfigurationBox.

> NOTE: An AV2SampleEntry may indirectly contain additional boxes, for example ‘colr when carried by the `LayerPropertyContainerBox` as defined in the draft of ISOBMFF 14496-12 10th Edition to describe the colour properties of non-base layers.

§ 3.2.2. AV2 Codec Configuration Box

The ***AV2CodecConfigurationBox*** carries the decoder initialisation data required to decode the AV2 samples of a track.

```
aligned(8) class AV2CodecConfigurationBox extends Box('av2C')
{
  unsigned int(8) reserved1 = 0;
  unsigned int(8) config_obus_count_minus1;
  for (i = 0; i < config_obus_count_minus1 + 1; ++i) {
    unsigned int(8) config_obu[];
  }
}
```

`reserved` SHALL be set to 0. Readers SHALL ignore its value.

`config_obus_count_minus1` is the number of `config_obu` minus one.

`config_obu` is a byte array carrying one OBU and its length as described in § 3.2.2.1 config_obu.

§ *3.2.2.1. config_obu*

Each `config_obu` SHALL carry a single OBU preceded by a `leb128() num_bytes_in_obu` value giving the byte length of that OBU, as specified in the Length delimited bitstream format from Annex B of [AV2].

The OBUs within an AV2CodecConfigurationBox SHALL appear in the order specified by the ordering of OBUs in a temporal unit in [AV2], with the following constraint:

- the OBUs within an AV2CodecConfigurationBox SHALL NOT contain OBUs of the types forbidden there as stated in Figure 3.1.

AV2 OBU type	Be present in `AV2CodecConfigurationBox` (in `config_obu`)	Be present in a sample
`OBU_SEQUENCE_HEADER`	**SHALL**	**SHALL NOT**
`OBU_TEMPORAL_DELIMITER`	**SHALL NOT**	**SHALL NOT** (the AV2 sample boundary already delimits the temporal unit)
`OBU_MULTI_FRAME_HEADER`	**SHALL NOT**	**MAY**
`OBU_CLOSED_LOOP_KEY`	**SHALL NOT**	**MAY**
`OBU_OPEN_LOOP_KEY`	**SHALL NOT**	**MAY**
`OBU_LEADING_TILE_GROUP`	**SHALL NOT**	**MAY**
`OBU_REGULAR_TILE_GROUP`	**SHALL NOT**	**MAY**
`OBU_METADATA_SHORT` or `OBU_METADATA_GROUP`, when its contents do not change for the duration of the active AV2SampleEntry	**MAY**	**SHOULD NOT**
`OBU_METADATA_SHORT` or `OBU_METADATA_GROUP`, otherwise	**SHOULD NOT**	**MAY**
`OBU_SWITCH`	**SHALL NOT**	**MAY**
`OBU_LEADING_SEF`	**SHALL NOT**	**MAY**
`OBU_REGULAR_SEF`	**SHALL NOT**	**MAY**
`OBU_LEADING_TIP`	**SHALL NOT**	**MAY**
`OBU_REGULAR_TIP`	**SHALL NOT**	**MAY**
`OBU_BUFFER_REMOVAL_TIMING`	**SHALL NOT**	**MAY**
`OBU_LAYER_CONFIGURATION_RECORD`	**MAY**	**SHALL NOT**
`OBU_ATLAS_SEGMENT`	**MAY**	**SHALL NOT**
`OBU_OPERATING_POINT_SET`	**MAY**	**SHALL NOT**
`OBU_BRIDGE_FRAME`	**SHALL NOT**	**MAY**
`OBU_MSDO`	**MAY**	**SHALL NOT**
`OBU_RAS_FRAME`	**SHALL NOT**	**MAY**
`OBU_QUANTIZATION_MATRIX`	**SHALL NOT**	**MAY**
	MAY	**SHOULD NOT**

`OBU_FILM_GRAIN`, when its contents do not change for the duration of the active AV2SampleEntry		
`OBU_FILM_GRAIN`, otherwise	**SHOULD NOT**	**MAY**
`OBU_CONTENT_INTERPRETATION`	**MAY**	**SHALL NOT**
`OBU_PADDING`	**SHALL NOT**	**SHALL NOT** (byte-level padding within a track is achieved using ISOBMFF mechanisms, e.g. 'free'/'skip' boxes or sample size signalling)

Figure 3.1: List of AV2 OBU types and where they can be found

ISSUE 1 Editor's note: for a Multistream bitstream, the AV2CodecConfigurationBox contains a Sequence Header OBU for each extended layer. The impact on multi-layer use cases — including how a reader determines the active decoder configuration per extended layer from multiple Sequence Header OBUs — needs investigation. Also how profile / level is signalled to decoder API for layers. etc.

3.3. AV2 Sample Format

3.3.1. General

The AV2 sample boundary delimits the temporal unit.

Each OBU within a sample SHALL be framed using the Length delimited bitstream format specified in Annex B of [AV2]: each OBU is preceded by a `leb128() num_bytes_in_obu` field giving the byte length of the OBU that follows. Figure 3.2 illustrates this mapping.

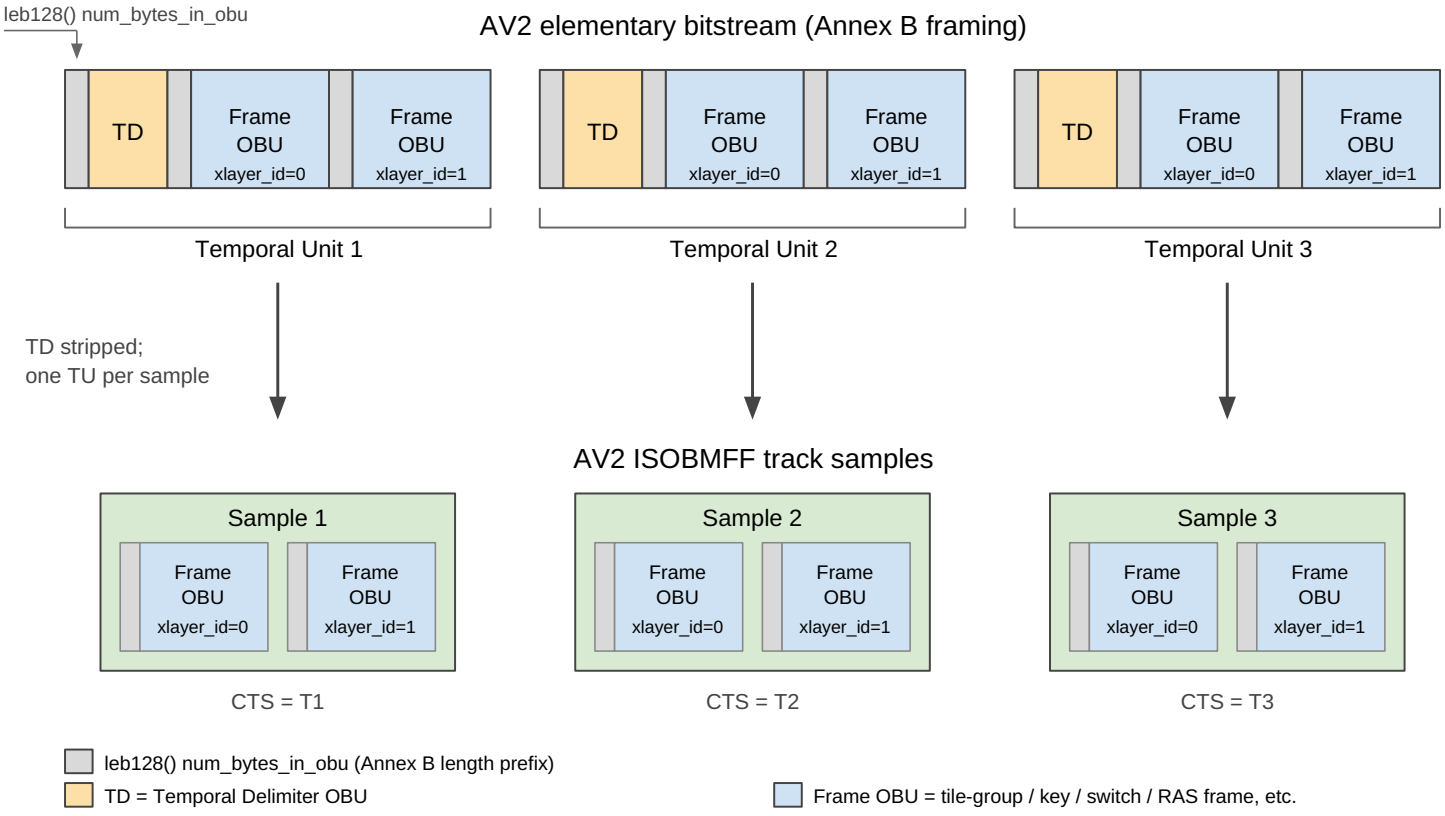


Figure 3.2: Example mapping of AV2 temporal units into the ISOBMFF sample format.

§ 3.3.2. OBUs in samples

The OBUs within a sample SHALL appear in the order specified by the ordering of OBUs in a temporal unit in [AV2], with the following constraint:

- the OBUs within a sample SHALL NOT contain OBUs of the types forbidden there as stated in Figure 3.1.

`OBU_CONTENT_INTERPRETATION` SHALL NOT be present in an AV2 bitstream conformant to this specification. The following container-level signalling is available to replace the semantics of the following `OBU_CONTENT_INTERPRETATION` fields:

- `ci_scan_type_idc`: ‘ilce sample group from ISO/IEC [23001-17]
- `ci_color_description_idc`, `ci_color_primaries`, `ci_transfer_characteristics`, `ci_matrix_coefficients`, and `ci_full_range_flag`: ‘colr box from [ISOBMFF] with the ‘nclx ‘colour_type
- `ci_chroma_sample_position_top`: ‘pixi box from [ISOBMFF]
- `ci_aspect_ratio_idc`, `ci_sar_width`, and `ci_sar_height`: ‘iso3 brand, ‘pasr sample group, or ‘pasp box from [ISOBMFF]
- `timing_info()`: ‘mvhd and ‘stts boxes from [ISOBMFF]

§ 3.3.3. Sync sample

An AV2 sample is a ***sync AV2 sample*** if, for every coded extended layer unit present in the AV2 sample, the first coded frame OBU of that coded extended layer unit has `obu_type` equal to `OBU_CLOSED_LOOP_KEY`. Such

an AV2 sample is a Closed Random Access point for the carried bitstream and corresponds to a SAP of type 1 or 2 as defined in Annex I of [ISOBMFF].

A sync AV2 sample SHALL be self-sufficient for starting decoding at that point for all layers: every OBU referenced by the coded frame OBUs of the sync AV2 sample SHALL be available either by being carried in the AV2CodecConfigurationBox of the active AV2SampleEntry or by being present in the sample, as required by the availability of HLS OBUs in [AV2].

> NOTE: In ISOBMFF terms, sync samples are signalled using the SyncSampleBox ('stss') in unfragmented tracks, or by setting `sample_is_non_sync_sample` equal to 0 in fragmented tracks. When every sample in a track is a sync sample, the SyncSampleBox is omitted in unfragmented tracks (per [ISOBMFF]), and in fragmented tracks this is expressed through the default sample flags of the MovieExtendsBox ('trex') or TrackFragmentHeaderBox ('tfhd') with no per-sample override.

§ 3.3.4. Layers

The following applies to all coded extended layer units for each temporal layer in an AV2 sample associated with a 'pixi box:

- The channels in the AV2 sample are considered to be ordered as if the layers were concatenated in increasing `xlayer_id` order, then in increasing `mlayer_id` order.
- If the 'pixi box describes the Nth channel as used and specified (`channel_idc` values other than 0 and 1), the Nth channel in the AV2 sample SHALL be present.
- If the 'pixi box describes the Nth channel to be a color channel (`channel_idc` values such as 2, 3, 4, and 7), the Nth channel in the AV2 sample SHALL be a color channel: if signalled, the `lcr_layer_type` field of its associated Layer Configuration Record OBU SHALL be set to 0 (`TEXTURE_LAYER`).
- If the 'pixi box describes the Nth channel to be an alpha channel (`channel_idc` 5), the Nth channel in the AV2 sample SHALL be an alpha channel: if signalled, the `lcr_layer_type` field of its associated Layer Configuration Record OBU SHALL be set to 1 (`AUX_LAYER`) and `lcr_auxiliary_type` to 0 (`ALPHA_AUX`).
- If the 'pixi box describes the Nth channel to be a depth channel (`channel_idc` 6), the Nth channel in the AV2 sample SHALL be a depth channel: if signalled, the `lcr_layer_type` field of its associated Layer Configuration Record OBU SHALL be set to 1 (`AUX_LAYER`) and `lcr_auxiliary_type` to 1 (`DEPTH_AUX`).
- If the 'pixi box describes the Nth channel as used and specified (`channel_idc` values other than 0 and 1) with `subsampling_flag` set to 1, the Nth channel in the AV2 sample SHALL have the value of the `chroma_format_idc` field of its associated Sequence Header OBU correspond to the same meaning as the value of the 'pixi box's Nth channel's `subsampling_type` value.

EXAMPLE 2 ¶

Simple example with an embedded layer containing YUV 4:2:0 colour channels and an embedded layer containing the gain map.

AV2 sample layer					**'pixi**	
xlayer_id	**lcr_chroma_format_idc**	**mlayer_id**	**lcr_layer_type**	**lcr_auxiliary_type**	**(i+1)th channel**	**ch**
0	CHROMA_FORMAT_420	0	TEXTURE_LAYER		0	ch mo
					1	ch **U**
					2	ch **V**
		1	AUX_LAYER	GAIN_MAP_AUX	3	U
					4	U
					5	U

EXAMPLE 3 ¶

Complex example with an extended layer containing CMYK channels (each channel as an embedded layer), an extended layer containing alpha, and an extended layer containing segmentation and depth data.

AV2 sample layer					'pixi	
xlayer_id	lcr_chroma_format_idc	mlayer_id	lcr_layer_type	lcr_auxiliary_type	(i+1)th channel	ch
0	CHROMA_FORMAT_400	0	TEXTURE_LAYER		0	ch mo
		1	TEXTURE_LAYER		1	ch U
		2	TEXTURE_LAYER		2	ch V
		3	TEXTURE_LAYER		3	ch
1	CHROMA_FORMAT_400	0	AUX_LAYER	ALPHA_AUX	4	
2	CHROMA_FORMAT_400	0	AUX_LAYER	SEGMENTATION_AUX	5	U
		1	AUX_LAYER	DEPTH_AUX	6	

§ 3.3.5. AV2 sample reconstruction

A byte sequence containing a conformant [AV2] Temporal Unit using the Length delimited bitstream format specified in Annex B of [AV2] **MAY** be reconstructed from the OBUs of an AV2 sample by applying the following steps:

1. Write a Temporal Delimiter OBU, preceded by its `leb128()` length prefix.
2. Write the OBUs carried in the AV2CodecConfigurationBox of the corresponding AV2SampleEntry, in the same order as present in the AV2CodecConfigurationBox, each preceded by its `leb128()` length prefix as present in the AV2CodecConfigurationBox.
3. Write the OBUs carried in the sample, in the same order as present in the sample, each preceded by its `leb128()` length prefix as present in the sample.

NOTE: AV2-ISOBMFF reader implementations do not have to reconstruct the contiguous byte sequence containing a conformant AV2 bitstream or Temporal Unit in memory upon decoding.

§ 4. CMAF AV2 track format

[CMAF] defines structural constraints on ISOBMFF files additional to [ISOBMFF] for the purpose of, for example, adaptive streaming or for protected files. Conformance to these structural constraints is signaled by the presence of the brands `cmfc` or `cmf2` in the `FileTypeBox`.

NOTE: It is important to note that while the `cmf2` brand does not introduce any new requirements compared to the previously defined `cmfc` brand, it further restricts the `cmfc` constraints.

For compatibility reasons, when the `cmf2` brand is used, the `cmfc` brand SHOULD also be included in the `FileTypeBox`.

If a CMAF Video Track uses the brand `av02`, it is called a ***CMAF AV2 Track*** and the following constraints, defining the ***CMAF Media Profile for AV2***, apply:

- it SHALL use an AV2SampleEntry, possibly transformed by encryption as specified in § 5 Common Encryption;
- it MAY use multiple sample entries, and in that case the following values SHALL not change in the track:
  - `seq_profile_idc`
  - `single_picture_header_flag`
  - `seq_level_idx`
  - `seq_tier`
  - `chroma_format_idc`
  - `bit_depth_idc`
  - `seq_initial_display_delay_minus_1`

When protected, CMAF AV2 Tracks SHALL use the signaling defined in [CMAF], which in turn relies on [CENC], with the provisions specified in § 5 Common Encryption.

§ 5. Common Encryption

CMAF AV2 Tracks and non-segmented AV2 files MAY be protected. If protected, they SHALL conform to [CENC]. Files SHOULD be protected using the `cbcs` protection scheme and MAY be protected using the `cenc` protection scheme.

When the protected scheme `cenc` is used, samples SHALL be protected using subsample encryption and SHALL NOT use pattern encryption.

When the protected scheme `cbcs` is used, samples SHALL be protected using subsample encryption and SHALL use pattern encryption. [CENC] recommends that the Block Pattern length be 10, i.e. `crypt_byte_block + skip_byte_block = 10`, where a Block is of size 16-bytes. In this specification, the

combination of `crypt_byte_block` and `skip_byte_block` SHALL be one of the following, with the first one being recommended:

- `crypt_byte_block = 1` and `skip_byte_block = 9`,
- `crypt_byte_block = 5` and `skip_byte_block = 5`,
- `crypt_byte_block = 10` and `skip_byte_block = 0`.

§ 5.1. General Subsample Encryption constraints

Within protected samples, the following constraints apply:

- Protected samples SHALL be exactly spanned by one or more contiguous subsamples.
  - An OBU MAY be spanned by one or more subsamples, especially when it has multiple ranges of protected data. However, as recommended in [CENC], subsample encryption entries **SHOULD** be as compactly represented as possible. For example, only one subsample encryption entry should be created for multiple consecutive unprotected OBUs as well as the first unprotected and protected parts of the following protected OBU, if such protected OBU exists.
  - A large unprotected OBU whose data size is larger than the maximum size of a single BytesOfClearData field MAY be spanned by multiple subsamples with zero size BytesOfProtectedData.
- All OBU Headers and associated obu_size fields SHALL be unprotected.
- Temporal Delimiter OBUs, Sequence Header OBUs, and Frame Header OBUs (including within a Frame OBU), and Redundant Frame Header OBUs SHALL be unprotected.
- Metadata OBUs SHALL NOT be protected.
- Tile Group OBUs and Frame OBUs SHALL be protected. Within Tile Group OBUs or Frame OBUs, the following applies:
  - for the protected scheme `cbcs`:
    - BytesOfProtectedData SHALL start on the first byte and end on the last byte of the `decode_tile` structure (including any trailing bits).

    > NOTE: As a result of the above, BytesOfProtectedData is not necessarily a multiple of 16 bytes and partial blocks are included in the BytesOfProtectedData range, but per [CENC] the data is not be encrypted.

    - A subsample SHALL be created for each tile, even if its size is less than 16 bytes.
    - All other parts of Tile Group OBUs and Frame OBUs SHALL be unprotected .
  - for the protected scheme `cenc`:
    - BytesOfProtectedData SHALL be a multiple of 16 bytes.
    - BytesOfProtectedData SHALL end on the last byte of the `decode_tile` structure (including any trailing bits).
    - BytesOfProtectedData SHALL span all complete 16-byte blocks of the `decode_tile` structure (including any trailing bits).

> NOTE: As a result of the above, partial blocks are not used and it is possible that BytesOfProtectedData does not start at the first byte of the `decode_tile` structure, but some number of bytes following that.

- A subsample SHALL be created for each tile that has a `decode_tile` structure whose size (including any trailing bits) is larger or equal to 16 bytes. If it is less than 16 bytes, per the rules above, the `decode_tile` structure is not encrypted and the corresponding bytes SHOULD be included in the BytesOfClearData field of a surrounding subsample, if any.
- All other parts of Tile Group OBUs and Frame OBUs SHALL be unprotected .

Version 1 and version 2 of the 'saiz box as defined in [ISOBMFF] **SHALL** be supported.

6. Codecs Parameter String

DASH and other applications require defined values for the 'Codecs parameter specified in [RFC6381] for ISO Media tracks. The codecs parameter string for the AOM AV2 codec is as follows:

```
<sample entry 4CC>.<profile>.<level><tier>.<bitDepth>.<monochrome>.<chromaSubsampling>.
<colorPrimaries>.<transferCharacteristics>.<matrixCoefficients>.<videoFullRangeFlag>
```

All fields following the sample entry 4CC are expressed as double digit decimals, unless indicated otherwise. Leading or trailing zeros cannot be omitted.

- The `profile` parameter value SHALL equal the value of seq_profile_idc in the first Sequence Header OBU.
- The `level` parameter value SHALL equal the level value indicated by seq_level_idx in the first Sequence Header OBU.
- The `tier` parameter value SHALL be equal to `M` when the seq_tier value in the first Sequence Header OBU is equal to 0, and `H` when it is equal to 1.
- The `bitDepth` parameter value SHALL equal the value of the `BitDepth` variable as defined in [AV2] derived from the first Sequence Header OBU.
- The `monochrome` parameter value, represented by `0` or `1`, SHALL equal the value of the `Monochrome` variable as defined in [AV2] derived from the first Sequence Header OBU.
- The `chromaSubsampling` parameter value, represented by a two-digit decimal:
  1. SHALL have its first digit equal to the `subsampling_type` value of the channels with `channel_idc` set to 3 (Second colour channel) and 4 (Third colour channel) in the 'pixi box associated with the AV2 sample if that `subsampling_type` value is signaled, and 0 otherwise.
  2. SHALL have its second digit equal to the `subsampling_location` value of the channels with `channel_idc` set to 3 (Second colour channel) and 4 (Third colour channel) in the 'pixi box associated with the AV2 sample if that `subsampling_location` value is signaled, and 0 otherwise.
- The `colorPrimaries`, `transferCharacteristics`, `matrixCoefficients`, and `videoFullRangeFlag` parameter values SHALL equal the values of matching fields in the 'colr box with the 'nclx 'colour_type.

> NOTE: A ‘colr box with the ‘nclx ‘colour_type is mandated to be present in § 3.2.1 AV2 Sample Entry.

- The `videoFullRangeFlag` is represented by `0` or `1`.

If the codecs parameter string ends with ".0.116.01.01.01.0" (containing all the default values below), that trailing part of the string SHOULD be omitted.

EXAMPLE 4 ¶

`codecs="av02.0.04M.10.0.22.09.16.09.0"` represents [AV2] `Main_420_10_IP0` profile at Interoperability Point 0, level 4.0, Main tier, 10-bit content, non-monochrome, 4:2:0 chroma subsampling co-located with (0, 0) luma sample, ITU-R BT.2100 color primaries, ITU-R BT.2100 PQ transfer characteristics, ITU-R BT. 2100 YCbCr color matrix, and studio swing representation.

The parameters `sample entry 4CC`, `profile`, `level`, `tier`, and `bitDepth` are all mandatory fields. If any of these fields is empty, or not within its allowed range, the processing device SHOULD treat it as an error. All the other fields (including their leading '.') are optional, mutually inclusive (all or none) fields. If not specified then the values listed in the table below are assumed.

mono_chrome	0
chromaSubsampling	22 (4:2:0 chroma subsampling co-located with (0, 0) luma sample)
colorPrimaries	01 (ITU-R BT.709)
transferCharacteristics	01 (ITU-R BT.709)
matrixCoefficients	01 (ITU-R BT.709)
videoFullRangeFlag	0 (studio swing representation)

EXAMPLE 5 ¶

The string codecs="av02.0.01M.08" in this case would represent [AV2] `Main_420_10_IP0` profile at Interoperability Point 0, level 2.1, Main tier, 8-bit content, 4:2:0 chroma subsampling co-located with (0, 0) luma sample, ITU-R BT.709 color primaries, transfer characteristics, matrix coefficients, and studio swing representation.

If any character that is not '.', digits, part of the AV2 4CC, or a tier value is encountered, the string **SHALL** be interpreted ignoring all the characters starting from that character.

§ Index

§ Terms defined by this specification

av02, in § 3.1

AV2CodecConfigurationBox, in § 3.2.2

AV2 sample, in § 3

AV2SampleEntry, in § 3.2.1

CMAF AV2 Track, in § 4

CMAF Media Profile for AV2, in § 4

sync AV2 sample, in § 3.3.3

§ Terms defined by reference

[23001-17] defines the following terms:

- ilce

[AV2] defines the following terms:

- AV2 bitstream
- availability of HLS OBUs
- Bridge Frame OBU
- Closed Loop Key OBU
- Closed Random Access
- coded extended layer unit
- Frame Header OBU
- High-Level Syntax
- Inter Frame
- Intra frame
- Key Frame
- Layer Configuration Record OBU
- Leading frame
- Length delimited bitstream format
- metadata group OBU syntax
- Metadata OBU
- metadata short OBU syntax
- Multistream
- OBU
- OBU Header
- obu_header_extension_flag
- obu_mlayer_id
- obu_size
- obu_tlayer_id
- obu_type
- obu_xlayer_id
- Open Loop Key OBU
- Open Random Access
- ordering of OBUs in a temporal unit
- sample
- sample value
- seq_level_idx
- seq_profile_idc
- seq_tier
- Sequence Header OBU
- show_existing_frame
- Singlestream
- Switch Frame
- Switch OBU
- Temporal Delimiter OBU
- Temporal Unit
- Tile Group OBU

[CENC] defines the following terms:

- BytesOfClearData
- BytesOfProtectedData
- cbcs
- cenc

[CMAF] defines the following terms:

- CMAF Video Track
- cmf2
- cmfc

[ISOBMFF] defines the following terms:

- btrt
- clli
- colour_type
- colr
- ctts
- iso3
- iso6
- mdcv
- mvhd
- nclx
- pasp
- pasr
- pixi
- saiz
- sample
- stts
- VisualSampleEntry

[RFC6381] defines the following terms:

codecs

§ References

§ Normative References

[23001-17]
Information technology — MPEG systems technologies — Part 17: Carriage of uncompressed video and images in ISO base media file format. Standard. URL: https://www.iso.org/standard/89120.html

[AV2]
AV2 Bitstream & Decoding Process Specification. Standard. URL: https://av2.aomedia.org/v1.0.0/index.html

[CENC]
Information technology — MPEG systems technologies — Part 7: Common encryption in ISO base media file format files. Standard. URL: https://www.iso.org/standard/84637.html

[CMAF]
Information technology — Multimedia application format (MPEG-A) — Part 19: Common media application format (CMAF) for segmented media. Standard. URL: https://www.iso.org/standard/71975.html

[ISOBMFF]
Information technology — Coding of audio-visual objects — Part 12: ISO base media file format. Standard. URL: https://www.iso.org/standard/95141.html

[RFC2119]
Key words for use in RFCs to Indicate Requirement Levels. Standard. URL: https://www.rfc-editor.org/info/rfc2119

[RFC6381]
R. Gellens; D. Singer; P. Frojdh. *The 'Codecs' and 'Profiles' Parameters for "Bucket" Media Types*. August 2011. Proposed Standard. URL: https://www.rfc-editor.org/info/rfc6381/

[RFC8174]
Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. Standard. URL: https://www.rfc-editor.org/info/rfc8174